



Department of Computer Science and Engineering
Midterm Examination Summer 2025
CSE 321: Operating Systems

Duration: 1 Hour 30 Minutes

Total Marks: 30

Answer all the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Sec:
-------	-----	------

1. Write the answers in the question paper. [1*2
=2]

CO1 a) Rushan wants to develop a server where everything that is being requested is handled by creating a child process of the original server process. The child process tends to complete and gets terminated, but after a while, Rushan notices that many inactive processes that have completed their task remain in the process table and are clogging up the system over time.

- I. **Identify** what type of issue Rushan is facing with her system.
- II. **State** how it can be solved.

Answer:

I	
II	

- b)** Observe the following code. **Calculate** how many processes and how many threads will be created. [1*2
=2]

```
main(){
    a = fork();
    b = fork();
    if (b > 0){
        pthread_create(...);
        fork();
    }
}
```

Answer:

Processes	
Threads	

c) A parent process forks, creating a child process. After the fork completes, the parent process creates a new shared memory segment and writes the number 111 into it. [1*2
=2]

- I. **Mention** in **yes** or **no** if the child process now attempts to access that same shared memory segment, will it be able to see the number 111?
- II. **Justify** the reason behind the previous answer.

Answer:

I	
II	

d) Find outputs of the given code:

[1*5
=5]

```
int main() {
    int counter = 0;
    int i;

    printf("Initial process starting. Counter: %d\n", counter);

    for (i = 0; i < 2; i++) {
        if (i == 0) {
            pid_t pid = fork();

            if (pid < 0) {
                printf("Error forking.\n");
                return 1;
            } else if (pid == 0) {
                counter += 10;
                printf("Child from loop %d: its counter is now %d.\n", i, counter);
                pid_t grandchild_pid = fork();
                if (grandchild_pid < 0) {
                    printf("Error forking grandchild.\n");
                    return 1;
                } else if (grandchild_pid == 0) {
                    counter += 100;
                    printf("Grandchild from loop %d: its counter is now %d.\n", i, counter);
                } else {
                    wait(NULL);
                    counter += 1;
                }
                break;
            } else {
                wait(NULL);
                counter += 1;
                printf("Parent after loop %d: its counter is now %d.\n", i, counter);
            }
        } else {
            printf("Process in second loop iteration %d: its counter is %d.\n", i, counter);
            break;
        }
    }

    return 0;
}
```

Write the outputs in the question paper. The output sequence should be exactly matched.

Outputs	
1	
2	
3	
4	
5	

2. a) Write the answers in the question paper.

[1*2

CO1 Suppose in a particular scenario FCFS CPU scheduler is being used.

=2]

- I. **What** significant drawback arises for short, I/O-bound processes when a long, CPU-intensive process is scheduled first using the algorithm?
- II. **How** does the issue lead to inefficient use of the CPU?

Answer:

I	
II	

b) Write the answer in the answer script.

Shoumo is implementing his own OS. For his OS, he has decided to develop a **Multilevel Feedback Queue** as a scheduler. To design the scheduler, he decided to divide the ready queue into **three** levels. According to his preferences, in every level round-robin algorithm will be used, and some constraints should be maintained in each level, which are given below.

- **Q0:** time quantum = 4, priority = 19
- **Q1:** time quantum = 6, priority = 25
- **Q2:** time quantum = 8, priority = 29

Here, the lower priority value indicates the higher priority, and in case of an I/O request, the scheduler will promote a process by one level upon I/O operation completion. At a certain moment, Shoumo has collected information on burst times, arrival times, and I/O times of three processes.

Process	Burst Time	Arrival Time	I/O Time
P1	3	2	3 [After 1s of total CPU allocation]
P2	21	0	3 [After 13s of total CPU allocation]
P3	14	7	N/A

- I. **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels. **[10]**
- II. **Calculate** the average waiting time and average turnaround time from the Gantt chart. **[1+1=2]**

3. a) Write the answers in the question paper.

[1*2

CO2 Consider the following code for the **test_and_set** hardware tool.

=2]

```
boolean test_and_set(boolean *target){
    boolean rv=*target;
    *target=true;
    return *target;
}

do{
    while(test_and_set(&lock))
        ; //do nothing

    //critical section

    lock=false;

    //remainder section
}while(true);
```

- I. **Identify** the error in the code that will create issues in synchronization.
- II. **Write** the fix for the error.

Answer:

I	
II	

b) Write the answers in the question paper.

[1]

In a mobile banking application, one thread handles balance inquiries as follows:

```
acquire(read_lock)
show_balance(account.balance)
release(read_lock)
```

Simultaneously, another thread processes cash-out transactions:

```
acquire(write_lock)
account.balance -= cashout_amount
release(write_lock)
```

As a user, Alex is logged in on two separate devices. From one device, he initiates a withdrawal, and almost instantly, from the other device, he checks the current balance. Occasionally, the displayed balance appears outdated or incorrect.

Identify the probable reason for this behavior.

Answer:

--

c) Write the answers in the question paper.

[1*2

Consider the following code. According to this code, **find out** what values Thread 1 and Thread 2 will print.

=2]

```
int array[] = {12, 67, 88, 34, 91, 5, 48, 73, 55, 29};

void* find_and_print_values(void* arg) {

    //if it is Thread 1 then print the elements in the first half of the array whose values are > 50
    //else print the elements in the second half of the array whose values are > 50

}

int main() {
    pthread_t threads[2];
    int i;

    for (i = 0; i < 2; i++) {
        pthread_create(&threads[i], NULL, find_and_print_values, (void*)(i + 1));
    }

    for (i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Answer:

Thread 1	
Thread 2	